

Consolidated_Modeling

October 6, 2026

1 Consolidated Modeling:

1.1 Loading Libraries and Preprocessing:

```
[1]: import os
import random
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.base import BaseEstimator
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

from sklearn.linear_model import LinearRegression, Ridge, LogisticRegression
from sklearn.ensemble import RandomForestRegressor, RandomForestClassifier

from sklearn.model_selection import (
    BaseCrossValidator,
    GridSearchCV
)

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score,
    accuracy_score,
    balanced_accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score
)
```

```

from xgboost import XGBRegressor, XGBClassifier

warnings.filterwarnings("ignore")

SEED = 42
np.random.seed(SEED)
random.seed(SEED)

print("Libraries loaded.")

```

Libraries loaded.

```

[2]: # Load your dataset
df = pd.read_csv("financial_regression.csv")

# Convert date
df["date"] = pd.to_datetime(df["date"])

# Sort chronologically
df = df.sort_values("date").reset_index(drop=True)

# Remove days where gold was not traded
df = df.dropna(subset=["gold close"]).reset_index(drop=True)

print("Dataset shape:", df.shape)
print("Date range:", df["date"].min(), "to", df["date"].max())

```

Dataset shape: (3719, 47)

Date range: 2010-01-14 00:00:00 to 2024-10-23 00:00:00

```

[3]: # -----
# Basic lagged features
# -----

df["Close_Lag1"] = df["gold close"].shift(1)
df["Open_Lag1"] = df["gold open"].shift(1)

# Today's open-to-close return
df["Daily_Return"] = (
    (df["gold close"] - df["gold open"])
    / df["gold open"]
)

# -----
# Calendar features
# -----

df["DayOfWeek"] = df["date"].dt.dayofweek

```

```

df["Month"] = df["date"].dt.month
df["Quarter"] = df["date"].dt.quarter
df["DayOfYear"] = df["date"].dt.dayofyear
df["IsMonthEnd"] = df["date"].dt.is_month_end.astype(int)

print("Basic features created.")

```

Basic features created.

```

[4]: # Tomorrow's gold closing price
df["Target_Next_Close"] = df["gold close"].shift(-1)

```

```

[5]: # =====
# ADVANCED LOG-RETURN FEATURES
# =====

target_assets = [
    "gold close",
    "silver close",
    "oil close",
    "sp500 close",
    "nasdaq close"
]

lags = [1, 2, 5, 10]

# -----
# Multi-day log returns
# -----

for asset in target_assets:

    col_name = asset.replace(" ", "_")

    for lag in lags:

        df[f"{col_name}_log_ret_{lag}d"] = np.log(
            df[asset] / df[asset].shift(lag)
        )

# -----
# Gold volatility
# -----

gold_ret = df["gold_close_log_ret_1d"]

df["gold_volatility_5d"] = (

```

```

    gold_ret.rolling(window=5).std()
)

df["gold_volatility_20d"] = (
    gold_ret.rolling(window=20).std()
)

# -----
# Gold distance from 20-day moving average
# -----

gold_sma20 = (
    df["gold close"]
    .rolling(window=20)
    .mean()
)

df["gold_sma20_dist"] = np.log(
    df["gold close"] / gold_sma20
)

# -----
# Gold/Silver ratio
# -----

df["gold_silver_ratio"] = (
    df["gold close"] / df["silver close"]
)

df["gold_silver_ratio_chg"] = np.log(
    df["gold_silver_ratio"]
    / df["gold_silver_ratio"].shift(1)
)

# -----
# Gold/Oil ratio
# -----

df["gold_oil_ratio"] = (
    df["gold close"] / df["oil close"]
)

df["gold_oil_ratio_chg"] = np.log(
    df["gold_oil_ratio"]

```

```

    / df["gold_oil_ratio"].shift(1)
)

# -----
# Gold volume change
# -----

df["gold_volume_change"] = np.log(
    (df["gold volume"] + 1)
    / (df["gold volume"].shift(1) + 1)
)

```

```

[6]: # -----
# Calendar / cyclical features
# -----

df["Month_sin"] = np.sin(
    2 * np.pi * df["Month"] / 12
)

df["Month_cos"] = np.cos(
    2 * np.pi * df["Month"] / 12
)

df["DayOfYear_sin"] = np.sin(
    2 * np.pi * df["DayOfYear"] / 365.25
)

df["DayOfYear_cos"] = np.cos(
    2 * np.pi * df["DayOfYear"] / 365.25
)

```

```

[7]: # =====
# NEXT-DAY LOG RETURN TARGET
# =====

df["Target_Log_Return"] = np.log(
    df["gold close"].shift(-1)
    / df["gold close"]
)

```

```

[ ]: # =====
# DIRECTIONAL TARGET
# =====

df["Target_Direction"] = (

```

```

    df["Target_Log_Return"] > 0
).astype(int)

print(
    df["Target_Direction"]
    .value_counts(normalize=True)
)

#1 = tomorrow's return is positive
#0 = tomorrow's return is zero or negative

```

```

Target_Direction
1    0.522452
0    0.477548
Name: proportion, dtype: float64

```

```

[9]: df = df.dropna(
      subset=[
          "Target_Next_Close",
          "Target_Log_Return"
      ]
    ).reset_index(drop=True)

print("Final dataset shape:", df.shape)

```

```

Final dataset shape: (3718, 90)

```

```

[10]: raw_numeric_features = [
      "sp500 open",
      "sp500 high",
      "sp500 low",
      "sp500 close",

      "nasdaq open",
      "nasdaq high",
      "nasdaq low",
      "nasdaq close",

      "silver close",
      "oil close",
      "platinum close",
      "palladium close",

      "gold open",
      "gold high",
      "gold low",
      "gold close",
      "gold volume",

```

```

        "Close_Lag1",
        "Open_Lag1",
        "Daily_Return"
    ]

    raw_categorical_features = [
        "DayOfWeek",
        "Month",
        "Quarter",
        "DayOfYear",
        "IsMonthEnd"
    ]

    raw_features = (
        raw_numeric_features
        + raw_categorical_features
    )

    raw_target = "Target_Next_Close"

```

```

[11]: log_numeric_features = [
        "Daily_Return",

        # 1-day returns
        "gold_close_log_ret_1d",
        "silver_close_log_ret_1d",
        "oil_close_log_ret_1d",
        "sp500_close_log_ret_1d",
        "nasdaq_close_log_ret_1d",

        # 2-day returns
        "gold_close_log_ret_2d",
        "silver_close_log_ret_2d",
        "oil_close_log_ret_2d",
        "sp500_close_log_ret_2d",
        "nasdaq_close_log_ret_2d",

        # 5-day returns
        "gold_close_log_ret_5d",
        "silver_close_log_ret_5d",
        "oil_close_log_ret_5d",
        "sp500_close_log_ret_5d",
        "nasdaq_close_log_ret_5d",

        # 10-day returns
        "gold_close_log_ret_10d",

```

```

"silver_close_log_ret_10d",
"oil_close_log_ret_10d",
"sp500_close_log_ret_10d",
"nasdaq_close_log_ret_10d",

# Volatility
"gold_volatility_5d",
"gold_volatility_20d",

# Trend
"gold_sma20_dist",

# Cross-market
"gold_silver_ratio_chg",
"gold_oil_ratio_chg",

# Volume
"gold_volume_change",

# Cyclical time
"Month_sin",
"Month_cos",
"DayOfYear_sin",
"DayOfYear_cos"
]

log_categorical_features = [
    "DayOfWeek",
    "IsMonthEnd"
]

log_features = (
    log_numeric_features
    + log_categorical_features
)

log_target = "Target_Log_Return"
classification_target = "Target_Direction"

```

```

[13]: class PurgedTimeSeriesCV(BaseCrossValidator):
        """
        Expanding-window time-series CV with a purge/embargo gap.

        Training observations always occur before validation observations.

        Parameters
        -----

```

```

n_splits : int
    Number of validation folds.

test_size : int
    Number of observations in each validation fold.

purge_size : int
    Number of observations removed immediately before
    the validation set.

embargo_size : int
    Additional gap between training and validation.
    For a strictly chronological walk-forward split,
    this acts as an additional gap before validation.
"""

def __init__(
    self,
    n_splits=3,
    test_size=200,
    purge_size=1,
    embargo_size=1
):
    self.n_splits = n_splits
    self.test_size = test_size
    self.purge_size = purge_size
    self.embargo_size = embargo_size

def split(self, X, y=None, groups=None):

    n_samples = len(X)

    # Total gap between train and validation
    gap = (
        self.purge_size
        + self.embargo_size
    )

    # Last possible validation fold
    first_test_start = (
        n_samples
        - self.n_splits * self.test_size
    )

    if first_test_start <= gap:
        raise ValueError(
            "Not enough observations for the "

```

```

        "requested number of splits."
    )

    for i in range(self.n_splits):

        test_start = (
            first_test_start
            + i * self.test_size
        )

        test_end = (
            test_start
            + self.test_size
        )

        train_end = test_start - gap

        train_indices = np.arange(
            0,
            train_end
        )

        test_indices = np.arange(
            test_start,
            min(test_end, n_samples)
        )

        yield train_indices, test_indices

    def get_n_splits(
        self,
        X=None,
        y=None,
        groups=None
    ):
        return self.n_splits

```

```

[ ]: # 2010-2013 → 2014
      # 2011-2014 → 2015
      # 2012-2015 → 2016
      # 2013-2016 → 2017
      # 2014-2017 → 2018
      # 2015-2018 → 2019
      # 2016-2019 → 2020
      # 2017-2020 → 2021
      # 2018-2021 → 2022
      # 2019-2022 → 2023

```

```
# 2020-2023 → 2024*
```

```
[14]: def generate_purged_walk_forward_windows(
    df,
    train_years=4,
    test_start_year=2014,
    test_end_year=2024,
    purge_size=1,
    embargo_size=1
):
    """
    Generate chronological walk-forward windows.

    Each fold uses:
        4 years training
        purge + embargo gap
        1 calendar year testing

    The final test year (2024) may be a partial year
    because the dataset ends on 2024-10-23.
    """

    windows = []

    for test_year in range(
        test_start_year,
        test_end_year + 1
    ):

        train_start = test_year - train_years
        train_end = test_year - 1

        train_start_date = pd.Timestamp(
            f"{train_start}-01-01"
        )

        train_end_date = pd.Timestamp(
            f"{train_end}-12-31"
        )

        test_start_date = pd.Timestamp(
            f"{test_year}-01-01"
        )

        test_end_date = pd.Timestamp(
            f"{test_year}-12-31"
        )
```

```

train_mask = (
    (df["date"] >= train_start_date)
    &
    (df["date"] <= train_end_date)
)

test_mask = (
    (df["date"] >= test_start_date)
    &
    (df["date"] <= test_end_date)
)

train_indices = np.where(train_mask)[0]
test_indices = np.where(test_mask)[0]

if len(train_indices) == 0 or len(test_indices) == 0:
    continue

# Total observations removed between
# training and testing
gap = purge_size + embargo_size

if len(train_indices) <= gap:
    continue

train_indices = train_indices[:-gap]

windows.append({
    "train_start": train_start,
    "train_end": train_end,
    "test_year": test_year,
    "train_indices": train_indices,
    "test_indices": test_indices
})

return windows

```

```

[15]: outer_windows = generate_purged_walk_forward_windows(
    df,
    train_years=4,
    test_start_year=2014,
    test_end_year=2024,
    purge_size=1,
    embargo_size=1
)

```

```

print(
    f"Generated {len(outer_windows)} "
    "purged walk-forward windows."
)

for w in outer_windows:

    train = df.iloc[w["train_indices"]]
    test = df.iloc[w["test_indices"]]

    print(
        f"Train: {train['date'].min().date()} "
        f"to {train['date'].max().date()} | "
        f"Test: {test['date'].min().date()} "
        f"to {test['date'].max().date()} | "
        f"Train n={len(train)}, "
        f"Test n={len(test)}"
    )

```

Generated 11 purged walk-forward windows.

```

Train: 2010-01-14 to 2013-12-27 | Test: 2014-01-02 to 2014-12-31 | Train n=996,
Test n=252
Train: 2011-01-03 to 2014-12-29 | Test: 2015-01-02 to 2015-12-31 | Train n=1004,
Test n=252
Train: 2012-01-03 to 2015-12-29 | Test: 2016-01-04 to 2016-12-30 | Train n=1004,
Test n=252
Train: 2013-01-02 to 2016-12-28 | Test: 2017-01-03 to 2017-12-29 | Train n=1006,
Test n=251
Train: 2014-01-02 to 2017-12-27 | Test: 2018-01-02 to 2018-12-31 | Train n=1005,
Test n=251
Train: 2015-01-02 to 2018-12-27 | Test: 2019-01-02 to 2019-12-31 | Train n=1004,
Test n=252
Train: 2016-01-04 to 2019-12-27 | Test: 2020-01-02 to 2020-12-31 | Train n=1004,
Test n=253
Train: 2017-01-03 to 2020-12-29 | Test: 2021-01-04 to 2021-12-31 | Train n=1005,
Test n=252
Train: 2018-01-02 to 2021-12-29 | Test: 2022-01-03 to 2022-12-30 | Train n=1006,
Test n=251
Train: 2019-01-02 to 2022-12-28 | Test: 2023-01-03 to 2023-12-29 | Train n=1006,
Test n=250
Train: 2020-01-02 to 2023-12-27 | Test: 2024-01-02 to 2024-10-22 | Train n=1004,
Test n=204

```

1.2 Phase 1:

```
[16]: def make_scaled_preprocessor(
    numeric_features,
    categorical_features
):

    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(
            strategy="median"
        )),
        ("scaler", StandardScaler())
    ])

    categorical_pipeline = Pipeline([
        ("imputer", SimpleImputer(
            strategy="most_frequent"
        )),
        ("onehot", OneHotEncoder(
            handle_unknown="ignore"
        ))
    ])

    return ColumnTransformer([
        (
            "numeric",
            numeric_pipeline,
            numeric_features
        ),
        (
            "categorical",
            categorical_pipeline,
            categorical_features
        )
    ])

```

```
[17]: def make_tree_preprocessor(
    numeric_features,
    categorical_features
):

    numeric_pipeline = Pipeline([
        ("imputer", SimpleImputer(
            strategy="median"
        ))
    ])

    categorical_pipeline = Pipeline([

```

```

        ("imputer", SimpleImputer(
            strategy="most_frequent"
        )),
        ("onehot", OneHotEncoder(
            handle_unknown="ignore"
        ))
    ])

    return ColumnTransformer([
        (
            "numeric",
            numeric_pipeline,
            numeric_features
        ),
        (
            "categorical",
            categorical_pipeline,
            categorical_features
        )
    ])

```

```
[18]: def regression_metrics(y_true, y_pred):
```

```

    return {
        "MAE": mean_absolute_error(
            y_true,
            y_pred
        ),

        "RMSE": np.sqrt(
            mean_squared_error(
                y_true,
                y_pred
            )
        ),

        "R2": r2_score(
            y_true,
            y_pred
        )
    }

```

```
[19]: RIDGE_GRID = {
        "model__alpha": np.logspace(
            -2,
            3,
            10

```

```
)  
}
```

```
[20]: RF_REG_GRID = {  
    "model__n_estimators": [  
        100,  
        200  
    ],  
    "model__max_depth": [  
        3,  
        5,  
        8  
    ],  
    "model__min_samples_leaf": [  
        2,  
        5,  
        10  
    ],  
    "model__max_features": [  
        "sqrt",  
        0.5  
    ]  
}
```

```
[21]: XGB_REG_GRID = {  
    "model__n_estimators": [  
        100,  
        200  
    ],  
    "model__max_depth": [  
        2,  
        3,  
        5  
    ],  
    "model__learning_rate": [  
        0.01,  
        0.05  
    ],  
    "model__subsample": [  
        0.7,  
        0.9  
    ]  
}
```

```

],

    "model__colsample_bytree": [
        0.7,
        0.9
    ],

    "model__reg_lambda": [
        1,
        10
    ]
}

```

```

[22]: def tune_model(
    pipeline,
    param_grid,
    X_train,
    y_train,
    cv,
    scoring="neg_root_mean_squared_error"
):

    search = GridSearchCV(
        estimator=pipeline,
        param_grid=param_grid,
        cv=cv,
        scoring=scoring,
        n_jobs=-1,
        refit=True
    )

    search.fit(
        X_train,
        y_train
    )

    return search

```

```

[23]: def build_linear_regression():

    preprocessor = make_scaled_preprocessor(
        raw_numeric_features,
        raw_categorical_features
    )

    return Pipeline([
        (

```

```

        "preprocessor",
        preprocessor
    ),
    (
        "model",
        LinearRegression()
    )
])

```

```

[24]: def build_ridge_regression():

    preprocessor = make_scaled_preprocessor(
        raw_numeric_features,
        raw_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            Ridge()
        )
    ])

```

```

[25]: def build_rf_regression():

    preprocessor = make_tree_preprocessor(
        raw_numeric_features,
        raw_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            RandomForestRegressor(
                random_state=SEED,
                n_jobs=-1
            )
        )
    ])

```

```
[26]: def build_xgb_regression():

    preprocessor = make_tree_preprocessor(
        raw_numeric_features,
        raw_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            XGBRegressor(
                objective="reg:squarederror",
                random_state=SEED,
                n_jobs=-1
            )
        )
    ])

```

```
[27]: raw_results = []

for w in outer_windows:

    train_idx = w["train_indices"]
    test_idx = w["test_indices"]

    train = df.iloc[train_idx].copy()
    test = df.iloc[test_idx].copy()

    X_train = train[raw_features]
    y_train = train[raw_target]

    X_test = test[raw_features]
    y_test = test[raw_target]

    # -----
    # Inner purged CV
    # -----

    inner_cv = PurgedTimeSeriesCV(
        n_splits=3,
        test_size=150,
        purge_size=1,
        embargo_size=1
    )

```

```

)

models = {

    "Linear Regression": (
        build_linear_regression(),
        None
    ),

    "Ridge Regression": (
        build_ridge_regression(),
        RIDGE_GRID
    ),

    "Random Forest": (
        build_rf_regression(),
        RF_REG_GRID
    ),

    "XGBoost": (
        build_xgb_regression(),
        XGB_REG_GRID
    )
}

for model_name, (pipeline, grid) in models.items():

    print(
        f"{w['test_year']} | "
        f"{model_name}"
    )

    # -----
    # Linear Regression has no hyperparameters
    # -----

    if grid is None:

        pipeline.fit(
            X_train,
            y_train
        )

        best_model = pipeline
        best_params = {}

    else:

```

```

        search = tune_model(
            pipeline,
            grid,
            X_train,
            y_train,
            inner_cv
        )

        best_model = search.best_estimator_
        best_params = search.best_params_

# -----
# Test
# -----

y_pred = best_model.predict(
    X_test
)

metrics = regression_metrics(
    y_test,
    y_pred
)

raw_results.append({
    "Test Year": w["test_year"],
    "Model": model_name,
    "MAE": metrics["MAE"],
    "RMSE": metrics["RMSE"],
    "R2": metrics["R2"],
    "Best Params": best_params
})

raw_results_df = pd.DataFrame(raw_results)

raw_results_df

```

```

2014 | Linear Regression
2014 | Ridge Regression
2014 | Random Forest
2014 | XGBoost
2015 | Linear Regression
2015 | Ridge Regression
2015 | Random Forest
2015 | XGBoost
2016 | Linear Regression
2016 | Ridge Regression

```

2016 | Random Forest
 2016 | XGBoost
 2017 | Linear Regression
 2017 | Ridge Regression
 2017 | Random Forest
 2017 | XGBoost
 2018 | Linear Regression
 2018 | Ridge Regression
 2018 | Random Forest
 2018 | XGBoost
 2019 | Linear Regression
 2019 | Ridge Regression
 2019 | Random Forest
 2019 | XGBoost
 2020 | Linear Regression
 2020 | Ridge Regression
 2020 | Random Forest
 2020 | XGBoost
 2021 | Linear Regression
 2021 | Ridge Regression
 2021 | Random Forest
 2021 | XGBoost
 2022 | Linear Regression
 2022 | Ridge Regression
 2022 | Random Forest
 2022 | XGBoost
 2023 | Linear Regression
 2023 | Ridge Regression
 2023 | Random Forest
 2023 | XGBoost
 2024 | Linear Regression
 2024 | Ridge Regression
 2024 | Random Forest
 2024 | XGBoost

[27]:

	Test Year	Model	MAE	RMSE	R^2	\
0	2014	Linear Regression	1.230866	1.614104	0.895138	
1	2014	Ridge Regression	0.945825	1.263190	0.935777	
2	2014	Random Forest	0.923618	1.237043	0.938408	
3	2014	XGBoost	0.959731	1.290856	0.932933	
4	2015	Linear Regression	1.358254	1.700766	0.908036	
5	2015	Ridge Regression	1.653868	1.931703	0.881366	
6	2015	Random Forest	3.583185	5.037881	0.193093	
7	2015	XGBoost	3.699582	5.218270	0.134274	
8	2016	Linear Regression	1.267696	1.701329	0.937492	
9	2016	Ridge Regression	1.113199	1.475792	0.952966	
10	2016	Random Forest	1.062635	1.421626	0.956356	

11	2016	XGBoost	1.961758	2.315398	0.884226
12	2017	Linear Regression	1.281449	1.643148	0.717402
13	2017	Ridge Regression	1.069307	1.318527	0.818032
14	2017	Random Forest	0.645623	0.811726	0.931034
15	2017	XGBoost	0.682360	0.860739	0.922454
16	2018	Linear Regression	0.807872	1.029629	0.961010
17	2018	Ridge Regression	0.621750	0.828651	0.974746
18	2018	Random Forest	0.629134	0.819117	0.975324
19	2018	XGBoost	0.670276	0.852313	0.973283
20	2019	Linear Regression	0.866627	1.144047	0.982370
21	2019	Ridge Regression	0.770092	1.009627	0.986270
22	2019	Random Forest	6.179332	8.447406	0.038825
23	2019	XGBoost	6.263809	8.589285	0.006267
24	2020	Linear Regression	2.407745	2.955414	0.948616
25	2020	Ridge Regression	2.173962	2.701252	0.957074
26	2020	Random Forest	22.903771	26.260479	-3.056931
27	2020	XGBoost	23.116795	26.492825	-3.129038
28	2021	Linear Regression	1.245187	1.657457	0.865693
29	2021	Ridge Regression	1.164199	1.573946	0.878886
30	2021	Random Forest	1.827534	2.277779	0.746348
31	2021	XGBoost	1.434459	1.902382	0.823066
32	2022	Linear Regression	1.660323	2.127189	0.941157
33	2022	Ridge Regression	1.530279	1.963371	0.949871
34	2022	Random Forest	1.362364	1.739204	0.960664
35	2022	XGBoost	1.673881	2.165564	0.939014
36	2023	Linear Regression	1.403912	1.833128	0.888424
37	2023	Ridge Regression	1.232137	1.622969	0.912541
38	2023	Random Forest	1.461751	1.985273	0.869134
39	2023	XGBoost	1.549464	2.095719	0.854169
40	2024	Linear Regression	1.925787	2.463783	0.981949
41	2024	Ridge Regression	2.252236	2.844625	0.975937
42	2024	Random Forest	26.956559	32.260474	-2.094810
43	2024	XGBoost	26.512068	31.807945	-2.008595

Best Params

```

0 {}
1 {'model__alpha': 5.994842503189409}
2 {'model__max_depth': 5, 'model__max_features':...
3 {'model__colsample_bytree': 0.9, 'model__learn...
4 {}
5 {'model__alpha': 21.544346900318846}
6 {'model__max_depth': 5, 'model__max_features':...
7 {'model__colsample_bytree': 0.9, 'model__learn...
8 {}
9 {'model__alpha': 1.6681005372000592}
10 {'model__max_depth': 5, 'model__max_features':...
11 {'model__colsample_bytree': 0.9, 'model__learn...
```

```

12                                     {}
13             {'model__alpha': 5.994842503189409}
14 {'model__max_depth': 8, 'model__max_features':...
15 {'model__colsample_bytree': 0.7, 'model__learn...
16                                     {}
17             {'model__alpha': 5.994842503189409}
18 {'model__max_depth': 8, 'model__max_features':...
19 {'model__colsample_bytree': 0.9, 'model__learn...
20                                     {}
21             {'model__alpha': 5.994842503189409}
22 {'model__max_depth': 5, 'model__max_features':...
23 {'model__colsample_bytree': 0.7, 'model__learn...
24                                     {}
25             {'model__alpha': 5.994842503189409}
26 {'model__max_depth': 8, 'model__max_features':...
27 {'model__colsample_bytree': 0.9, 'model__learn...
28                                     {}
29             {'model__alpha': 5.994842503189409}
30 {'model__max_depth': 5, 'model__max_features':...
31 {'model__colsample_bytree': 0.7, 'model__learn...
32                                     {}
33             {'model__alpha': 0.464158883361278}
34 {'model__max_depth': 8, 'model__max_features':...
35 {'model__colsample_bytree': 0.9, 'model__learn...
36                                     {}
37             {'model__alpha': 5.994842503189409}
38 {'model__max_depth': 8, 'model__max_features':...
39 {'model__colsample_bytree': 0.7, 'model__learn...
40                                     {}
41             {'model__alpha': 77.42636826811278}
42 {'model__max_depth': 5, 'model__max_features':...
43 {'model__colsample_bytree': 0.7, 'model__learn...

```

```

[28]: naive_raw_results = []

for w in outer_windows:

    test = df.iloc[
        w["test_indices"]
    ]

    y_true = test["Target_Next_Close"]

    # Today's close predicts tomorrow's close
    y_pred = test["gold close"].values

    metrics = regression_metrics(

```

```

        y_true,
        y_pred
    )

    naive_raw_results.append({
        "Test Year": w["test_year"],
        "Model": "Naive - Today's Close",
        "MAE": metrics["MAE"],
        "RMSE": metrics["RMSE"],
        "R2": metrics["R2"],
        "Best Params": {}
    })

naive_raw_df = pd.DataFrame(
    naive_raw_results
)

raw_results_df = pd.concat(
    [
        raw_results_df,
        naive_raw_df
    ],
    ignore_index=True
)

```

```

[29]: raw_summary = (
    raw_results_df
    .groupby("Model")
    [["MAE", "RMSE", "R2"]]
    .mean()
    .sort_values("RMSE")
)

raw_summary.round(4)

```

```

[29]:

```

	MAE	RMSE	R ²
Model			
Naive - Today's Close	0.9798	1.3100	0.9587
Ridge Regression	1.3206	1.6849	0.9294
Linear Regression	1.4051	1.8064	0.9116
Random Forest	6.1396	7.4816	0.1325
XGBoost	6.2295	7.5992	0.1211

1.3 Phase 2:

```
[30]: def build_linear_log():

    preprocessor = make_scaled_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            LinearRegression()
        )
    ])

def build_ridge_log():

    preprocessor = make_scaled_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            Ridge()
        )
    ])

def build_rf_log():

    preprocessor = make_tree_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    return Pipeline([
```

```

        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            RandomForestRegressor(
                random_state=SEED,
                n_jobs=-1
            )
        )
    ])

```

```

def build_xgb_log():

    preprocessor = make_tree_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            XGBRegressor(
                objective="reg:squarederror",
                random_state=SEED,
                n_jobs=-1
            )
        )
    ])

```

```

[31]: log_results = []

for w in outer_windows:

    train_idx = w["train_indices"]
    test_idx = w["test_indices"]

    train = df.iloc[train_idx].copy()
    test = df.iloc[test_idx].copy()

    X_train = train[log_features]

```

```

y_train = train[log_target]

X_test = test[log_features]
y_test = test[log_target]

inner_cv = PurgedTimeSeriesCV(
    n_splits=3,
    test_size=150,
    purge_size=1,
    embargo_size=1
)

models = {

    "Linear Regression": (
        build_linear_log(),
        None
    ),

    "Ridge Regression": (
        build_ridge_log(),
        RIDGE_GRID
    ),

    "Random Forest": (
        build_rf_log(),
        RF_REG_GRID
    ),

    "XGBoost": (
        build_xgb_log(),
        XGB_REG_GRID
    )
}

for model_name, (pipeline, grid) in models.items():

    print(
        f"{w['test_year']} | "
        f"{model_name}"
    )

    if grid is None:

        pipeline.fit(
            X_train,
            y_train

```

```

    )

    best_model = pipeline
    best_params = {}

else:

    search = tune_model(
        pipeline,
        grid,
        X_train,
        y_train,
        inner_cv
    )

    best_model = search.best_estimator_
    best_params = search.best_params_

y_pred = best_model.predict(
    X_test
)

metrics = regression_metrics(
    y_test,
    y_pred
)

log_results.append({
    "Test Year": w["test_year"],
    "Model": model_name,
    "MAE": metrics["MAE"],
    "RMSE": metrics["RMSE"],
    "R2": metrics["R2"],
    "Best Params": best_params
})

log_results_df = pd.DataFrame(
    log_results
)

```

2014 | Linear Regression
 2014 | Ridge Regression
 2014 | Random Forest
 2014 | XGBoost
 2015 | Linear Regression
 2015 | Ridge Regression
 2015 | Random Forest
 2015 | XGBoost

```

2016 | Linear Regression
2016 | Ridge Regression
2016 | Random Forest
2016 | XGBoost
2017 | Linear Regression
2017 | Ridge Regression
2017 | Random Forest
2017 | XGBoost
2018 | Linear Regression
2018 | Ridge Regression
2018 | Random Forest
2018 | XGBoost
2019 | Linear Regression
2019 | Ridge Regression
2019 | Random Forest
2019 | XGBoost
2020 | Linear Regression
2020 | Ridge Regression
2020 | Random Forest
2020 | XGBoost
2021 | Linear Regression
2021 | Ridge Regression
2021 | Random Forest
2021 | XGBoost
2022 | Linear Regression
2022 | Ridge Regression
2022 | Random Forest
2022 | XGBoost
2023 | Linear Regression
2023 | Ridge Regression
2023 | Random Forest
2023 | XGBoost
2024 | Linear Regression
2024 | Ridge Regression
2024 | Random Forest
2024 | XGBoost

```

```
[34]: log_results_df
```

```

[34]:   Test Year      Model      MAE      RMSE      R2 \
0      2014  Linear Regression  0.006804  0.009426 -0.021217
1      2014   Ridge Regression  0.006654  0.009319  0.001837
2      2014   Random Forest    0.006640  0.009332 -0.001019
3      2014      XGBoost        0.006631  0.009351 -0.005026
4      2015  Linear Regression  0.007157  0.009248 -0.080847
5      2015   Ridge Regression  0.006815  0.008937 -0.009437
6      2015   Random Forest    0.006951  0.009065 -0.038668

```

7	2015	XGBoost	0.007095	0.009199	-0.069506
8	2016	Linear Regression	0.008049	0.010729	-0.122932
9	2016	Ridge Regression	0.007500	0.010185	-0.012002
10	2016	Random Forest	0.007458	0.010146	-0.004251
11	2016	XGBoost	0.007471	0.010188	-0.012578
12	2017	Linear Regression	0.005421	0.006669	-0.140377
13	2017	Ridge Regression	0.005193	0.006416	-0.055642
14	2017	Random Forest	0.005384	0.006720	-0.157791
15	2017	XGBoost	0.005236	0.006517	-0.088981
16	2018	Linear Regression	0.004624	0.006174	-0.032004
17	2018	Ridge Regression	0.004501	0.006080	-0.000897
18	2018	Random Forest	0.004490	0.006103	-0.008465
19	2018	XGBoost	0.004486	0.006105	-0.009129
20	2019	Linear Regression	0.005785	0.007579	-0.067384
21	2019	Ridge Regression	0.005625	0.007426	-0.024869
22	2019	Random Forest	0.005581	0.007368	-0.008799
23	2019	XGBoost	0.005575	0.007368	-0.008698
24	2020	Linear Regression	0.009492	0.013805	-0.251126
25	2020	Ridge Regression	0.009003	0.012655	-0.051284
26	2020	Random Forest	0.008910	0.012509	-0.027145
27	2020	XGBoost	0.008882	0.012473	-0.021396
28	2021	Linear Regression	0.006566	0.008823	-0.060697
29	2021	Ridge Regression	0.006362	0.008632	-0.015337
30	2021	Random Forest	0.006338	0.008602	-0.008344
31	2021	XGBoost	0.006266	0.008512	0.012669
32	2022	Linear Regression	0.007674	0.009827	-0.055700
33	2022	Ridge Regression	0.007493	0.009657	-0.019446
34	2022	Random Forest	0.007394	0.009581	-0.003409
35	2022	XGBoost	0.007413	0.009596	-0.006599
36	2023	Linear Regression	0.006387	0.008494	-0.024511
37	2023	Ridge Regression	0.006329	0.008443	-0.012368
38	2023	Random Forest	0.006300	0.008385	0.001562
39	2023	XGBoost	0.006279	0.008371	0.004826
40	2024	Linear Regression	0.007083	0.009040	0.000214
41	2024	Ridge Regression	0.007085	0.009123	-0.018167
42	2024	Random Forest	0.007031	0.009087	-0.010259
43	2024	XGBoost	0.007040	0.009096	-0.012095
44	2014	Naive - Zero Return	0.006637	0.009328	-0.000207
45	2015	Naive - Zero Return	0.006835	0.008904	-0.002121
46	2016	Naive - Zero Return	0.007465	0.010129	-0.000776
47	2017	Naive - Zero Return	0.005022	0.006265	-0.006336
48	2018	Naive - Zero Return	0.004515	0.006079	-0.000413
49	2019	Naive - Zero Return	0.005579	0.007367	-0.008552
50	2020	Naive - Zero Return	0.008825	0.012377	-0.005729
51	2021	Naive - Zero Return	0.006280	0.008573	-0.001370
52	2022	Naive - Zero Return	0.007415	0.009565	-0.000045
53	2023	Naive - Zero Return	0.006287	0.008403	-0.002689

54 2024 Naive - Zero Return 0.007085 0.009141 -0.022090

```

Best Params
0 {}
1 {'model__alpha': 1000.0}
2 {'model__max_depth': 3, 'model__max_features':...
3 {'model__colsample_bytree': 0.9, 'model__learn...
4 {}
5 {'model__alpha': 1000.0}
6 {'model__max_depth': 5, 'model__max_features':...
7 {'model__colsample_bytree': 0.7, 'model__learn...
8 {}
9 {'model__alpha': 1000.0}
10 {'model__max_depth': 3, 'model__max_features':...
11 {'model__colsample_bytree': 0.7, 'model__learn...
12 {}
13 {'model__alpha': 1000.0}
14 {'model__max_depth': 8, 'model__max_features':...
15 {'model__colsample_bytree': 0.9, 'model__learn...
16 {}
17 {'model__alpha': 1000.0}
18 {'model__max_depth': 3, 'model__max_features':...
19 {'model__colsample_bytree': 0.7, 'model__learn...
20 {}
21 {'model__alpha': 1000.0}
22 {'model__max_depth': 3, 'model__max_features':...
23 {'model__colsample_bytree': 0.7, 'model__learn...
24 {}
25 {'model__alpha': 1000.0}
26 {'model__max_depth': 3, 'model__max_features':...
27 {'model__colsample_bytree': 0.7, 'model__learn...
28 {}
29 {'model__alpha': 1000.0}
30 {'model__max_depth': 3, 'model__max_features':...
31 {'model__colsample_bytree': 0.9, 'model__learn...
32 {}
33 {'model__alpha': 1000.0}
34 {'model__max_depth': 3, 'model__max_features':...
35 {'model__colsample_bytree': 0.7, 'model__learn...
36 {}
37 {'model__alpha': 1000.0}
38 {'model__max_depth': 5, 'model__max_features':...
39 {'model__colsample_bytree': 0.7, 'model__learn...
40 {}
41 {'model__alpha': 1000.0}
42 {'model__max_depth': 3, 'model__max_features':...
43 {'model__colsample_bytree': 0.9, 'model__learn...
```

```

44                                     {}
45                                     {}
46                                     {}
47                                     {}
48                                     {}
49                                     {}
50                                     {}
51                                     {}
52                                     {}
53                                     {}
54                                     {}

```

```

[32]: zero_return_results = []

for w in outer_windows:

    test = df.iloc[
        w["test_indices"]
    ]

    y_true = test["Target_Log_Return"]

    # Predict zero return
    y_pred = np.zeros(
        len(test)
    )

    metrics = regression_metrics(
        y_true,
        y_pred
    )

    zero_return_results.append({
        "Test Year": w["test_year"],
        "Model": "Naive - Zero Return",
        "MAE": metrics["MAE"],
        "RMSE": metrics["RMSE"],
        "R2": metrics["R2"],
        "Best Params": {}
    })

zero_return_df = pd.DataFrame(
    zero_return_results
)

log_results_df = pd.concat(
    [

```

```

        log_results_df,
        zero_return_df
    ],
    ignore_index=True
)

```

```

[33]: log_summary = (
        log_results_df
        .groupby("Model")
        [["MAE", "RMSE", "R2"]]
        .mean()
        .sort_values("RMSE")
    )

log_summary.round(6)

```

```

[33]:

```

	MAE	RMSE	R ²
Model			
Naive - Zero Return	0.006540	0.008739	-0.004575
XGBoost	0.006580	0.008798	-0.019683
Ridge Regression	0.006596	0.008807	-0.019783
Random Forest	0.006589	0.008809	-0.024235
Linear Regression	0.006822	0.009074	-0.077871

```

[35]: # Example:
# predicted return = r_hat

# predicted next close:
#
#  $P_{\text{hat}}(t+1) = P(t) * \exp(r_{\text{hat}})$ 

def return_to_price(
    current_price,
    predicted_return
):

    return (
        current_price
        * np.exp(predicted_return)
    )

```

```

[36]: predicted_price = return_to_price(
        test["gold close"].values,
        y_pred
    )

```

```

[37]: predicted_price[:5], test["Target_Next_Close"].values[:5]

```

```
[37]: (array([190.72, 189.13, 189.32, 189.35, 187.87]),
      array([189.13, 189.32, 189.35, 187.87, 187.93]))
```

1.4 Phase 3:

```
[38]: def build_logistic_classifier():

    preprocessor = make_scaled_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    model = LogisticRegression(
        solver="saga",
        penalty="elasticnet",
        max_iter=5000,
        random_state=SEED
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            model
        )
    ])
```

```
[39]: LOGISTIC_GRID = {

    "model__C": [
        0.001,
        0.01,
        0.1,
        1,
        10
    ],

    "model__l1_ratio": [
        0.1,
        0.5,
        0.9
    ]
}
```

```
[40]: def build_rf_classifier():

    preprocessor = make_tree_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            RandomForestClassifier(
                random_state=SEED,
                n_jobs=-1
            )
        )
    ])

```

```
[41]: RF_CLF_GRID = {

    "model__n_estimators": [
        100,
        200
    ],

    "model__max_depth": [
        2,
        3,
        5,
        8
    ],

    "model__min_samples_leaf": [
        5,
        10,
        20
    ],

    "model__max_features": [
        "sqrt",
        0.5
    ]
}

```

```
[42]: def build_xgb_classifier():

    preprocessor = make_tree_preprocessor(
        log_numeric_features,
        log_categorical_features
    )

    return Pipeline([
        (
            "preprocessor",
            preprocessor
        ),
        (
            "model",
            XGBClassifier(
                objective="binary:logistic",
                eval_metric="logloss",
                random_state=SEED,
                n_jobs=-1
            )
        )
    ])

```

```
[43]: XGB_CLF_GRID = {

    "model__n_estimators": [
        100,
        200
    ],

    "model__max_depth": [
        2,
        3,
        5
    ],

    "model__learning_rate": [
        0.01,
        0.05
    ],

    "model__subsample": [
        0.7,
        0.9
    ],

    "model__colsample_bytree": [

```

```

        0.7,
        0.9
    ],

    "model__reg_lambda": [
        1,
        10
    ]
}

```

```

[44]: def classification_metrics(
    y_true,
    y_pred,
    y_prob
):

    return {

        "Accuracy": accuracy_score(
            y_true,
            y_pred
        ),

        "Balanced Accuracy": balanced_accuracy_score(
            y_true,
            y_pred
        ),

        "Precision": precision_score(
            y_true,
            y_pred,
            zero_division=0
        ),

        "Recall": recall_score(
            y_true,
            y_pred,
            zero_division=0
        ),

        "F1": f1_score(
            y_true,
            y_pred,
            zero_division=0
        ),

        "ROC-AUC": roc_auc_score(

```

```

        y_true,
        y_prob
    )
}

```

```

[45]: def tune_classifier(
    pipeline,
    param_grid,
    X_train,
    y_train,
    cv
):

    search = GridSearchCV(
        estimator=pipeline,
        param_grid=param_grid,
        cv=cv,
        scoring="roc_auc",
        n_jobs=-1,
        refit=True
    )

    search.fit(
        X_train,
        y_train
    )

    return search

```

```

[46]: classification_results = []

for w in outer_windows:

    train_idx = w["train_indices"]
    test_idx = w["test_indices"]

    train = df.iloc[train_idx].copy()
    test = df.iloc[test_idx].copy()

    X_train = train[log_features]
    y_train = train[classification_target]

    X_test = test[log_features]
    y_test = test[classification_target]

    inner_cv = PurgedTimeSeriesCV(
        n_splits=3,

```

```

        test_size=150,
        purge_size=1,
        embargo_size=1
    )

models = {

    "Logistic Regression": (
        build_logistic_classifier(),
        LOGISTIC_GRID
    ),

    "Random Forest": (
        build_rf_classifier(),
        RF_CLF_GRID
    ),

    "XGBoost": (
        build_xgb_classifier(),
        XGB_CLF_GRID
    )
}

for model_name, (pipeline, grid) in models.items():

    print(
        f"{w['test_year']}] | "
        f"{model_name}"
    )

    search = tune_classifier(
        pipeline,
        grid,
        X_train,
        y_train,
        inner_cv
    )

    best_model = search.best_estimator_

    # Predictions
    y_pred = best_model.predict(
        X_test
    )

    # Probability of class 1
    y_prob = best_model.predict_proba(

```

```

        X_test
    )[:, 1]

    metrics = classification_metrics(
        y_test,
        y_pred,
        y_prob
    )

    classification_results.append({

        "Test Year": w["test_year"],

        "Model": model_name,

        "Accuracy": metrics["Accuracy"],

        "Balanced Accuracy":
            metrics["Balanced Accuracy"],

        "Precision":
            metrics["Precision"],

        "Recall":
            metrics["Recall"],

        "F1":
            metrics["F1"],

        "ROC-AUC":
            metrics["ROC-AUC"],

        "Best Params":
            search.best_params_
    })

classification_results_df = pd.DataFrame(
    classification_results
)

```

2014 | Logistic Regression
 2014 | Random Forest
 2014 | XGBoost
 2015 | Logistic Regression
 2015 | Random Forest
 2015 | XGBoost
 2016 | Logistic Regression
 2016 | Random Forest

```

2016 | XGBoost
2017 | Logistic Regression
2017 | Random Forest
2017 | XGBoost
2018 | Logistic Regression
2018 | Random Forest
2018 | XGBoost
2019 | Logistic Regression
2019 | Random Forest
2019 | XGBoost
2020 | Logistic Regression
2020 | Random Forest
2020 | XGBoost
2021 | Logistic Regression
2021 | Random Forest
2021 | XGBoost
2022 | Logistic Regression
2022 | Random Forest
2022 | XGBoost
2023 | Logistic Regression
2023 | Random Forest
2023 | XGBoost
2024 | Logistic Regression
2024 | Random Forest
2024 | XGBoost

```

```
[47]: classification_results_df
```

```

[47]:      Test Year      Model  Accuracy  Balanced Accuracy  Precision  \
0      2014  Logistic Regression  0.507937      0.515259  0.494565
1      2014      Random Forest  0.492063      0.498613  0.483146
2      2014      XGBoost  0.488095      0.491488  0.477124
3      2015  Logistic Regression  0.492063      0.518049  0.460317
4      2015      Random Forest  0.460317      0.490100  0.442211
5      2015      XGBoost  0.452381      0.475457  0.431694
6      2016  Logistic Regression  0.519841      0.523690  0.553846
7      2016      Random Forest  0.523810      0.526966  0.552632
8      2016      XGBoost  0.515873      0.516507  0.525862
9      2017  Logistic Regression  0.430279      0.497734  0.500000
10     2017      Random Forest  0.430279      0.483003  0.500000
11     2017      XGBoost  0.470120      0.515702  0.613636
12     2018  Logistic Regression  0.513944      0.514095  0.511111
13     2018      Random Forest  0.466135      0.465683  0.453608
14     2018      XGBoost  0.474104      0.473873  0.468468
15     2019  Logistic Regression  0.500000      0.497712  0.545455
16     2019      Random Forest  0.484127      0.475591  0.527027
17     2019      XGBoost  0.496032      0.489512  0.538462

```

18	2020	Logistic Regression	0.509881	0.483705	0.576687
19	2020	Random Forest	0.486166	0.465023	0.561290
20	2020	XGBoost	0.505929	0.499226	0.588235
21	2021	Logistic Regression	0.500000	0.500000	0.500000
22	2021	Random Forest	0.523810	0.523810	0.515306
23	2021	XGBoost	0.460317	0.460317	0.469880
24	2022	Logistic Regression	0.509960	0.500000	0.509960
25	2022	Random Forest	0.478088	0.473990	0.491525
26	2022	XGBoost	0.494024	0.492156	0.503356
27	2023	Logistic Regression	0.508000	0.500000	0.508000
28	2023	Random Forest	0.544000	0.540042	0.534759
29	2023	XGBoost	0.520000	0.518213	0.522876
30	2024	Logistic Regression	0.558824	0.500000	0.558824
31	2024	Random Forest	0.622549	0.600292	0.629371
32	2024	XGBoost	0.563725	0.548830	0.596899

	Recall	F1	ROC-AUC \
0	0.745902	0.594771	0.499811
1	0.704918	0.573333	0.458953
2	0.598361	0.530909	0.464250
3	0.769912	0.576159	0.536194
4	0.778761	0.564103	0.492265
5	0.699115	0.533784	0.478958
6	0.281250	0.373057	0.517956
7	0.328125	0.411765	0.504032
8	0.476562	0.500000	0.514365
9	0.013986	0.027211	0.581456
10	0.104895	0.173410	0.498511
11	0.188811	0.288770	0.538591
12	0.552000	0.530769	0.517841
13	0.352000	0.396396	0.492952
14	0.416000	0.440678	0.502921
15	0.521739	0.533333	0.490847
16	0.565217	0.545455	0.471205
17	0.557971	0.548043	0.460272
18	0.630872	0.602564	0.471154
19	0.583893	0.572368	0.500968
20	0.536913	0.561404	0.512584
21	1.000000	0.666667	0.500000
22	0.801587	0.627329	0.479781
23	0.619048	0.534247	0.459499
24	1.000000	0.675462	0.500000
25	0.679688	0.570492	0.471672
26	0.585938	0.541516	0.506479
27	1.000000	0.673740	0.500000
28	0.787402	0.636943	0.553038
29	0.629921	0.571429	0.552653

```

30 1.000000 0.716981 0.500000
31 0.789474 0.700389 0.606238
32 0.675439 0.633745 0.594055

```

Best Params

```

0      {'model__C': 1, 'model__l1_ratio': 0.1}
1  {'model__max_depth': 8, 'model__max_features':...
2  {'model__colsample_bytree': 0.9, 'model__learn...
3      {'model__C': 0.01, 'model__l1_ratio': 0.1}
4  {'model__max_depth': 2, 'model__max_features':...
5  {'model__colsample_bytree': 0.7, 'model__learn...
6      {'model__C': 0.01, 'model__l1_ratio': 0.1}
7  {'model__max_depth': 5, 'model__max_features':...
8  {'model__colsample_bytree': 0.7, 'model__learn...
9      {'model__C': 0.1, 'model__l1_ratio': 0.9}
10 {'model__max_depth': 8, 'model__max_features':...
11 {'model__colsample_bytree': 0.7, 'model__learn...
12      {'model__C': 0.1, 'model__l1_ratio': 0.5}
13 {'model__max_depth': 5, 'model__max_features':...
14 {'model__colsample_bytree': 0.7, 'model__learn...
15      {'model__C': 1, 'model__l1_ratio': 0.1}
16 {'model__max_depth': 3, 'model__max_features':...
17 {'model__colsample_bytree': 0.9, 'model__learn...
18      {'model__C': 0.1, 'model__l1_ratio': 0.5}
19 {'model__max_depth': 5, 'model__max_features':...
20 {'model__colsample_bytree': 0.7, 'model__learn...
21      {'model__C': 0.001, 'model__l1_ratio': 0.1}
22 {'model__max_depth': 8, 'model__max_features':...
23 {'model__colsample_bytree': 0.7, 'model__learn...
24      {'model__C': 0.001, 'model__l1_ratio': 0.1}
25 {'model__max_depth': 8, 'model__max_features':...
26 {'model__colsample_bytree': 0.7, 'model__learn...
27      {'model__C': 0.001, 'model__l1_ratio': 0.1}
28 {'model__max_depth': 8, 'model__max_features':...
29 {'model__colsample_bytree': 0.9, 'model__learn...
30      {'model__C': 0.001, 'model__l1_ratio': 0.1}
31 {'model__max_depth': 8, 'model__max_features':...
32 {'model__colsample_bytree': 0.9, 'model__learn...

```

```

[48]: majority_results = []

for w in outer_windows:

    train = df.iloc[
        w["train_indices"]
    ]

```

```

test = df.iloc[
    w["test_indices"]
]

y_train = train[
    classification_target
]

y_test = test[
    classification_target
]

# Determine majority class from training only
majority_class = (
    y_train
    .value_counts()
    .idxmax()
)

y_pred = np.full(
    len(y_test),
    majority_class
)

# Probability approximation
class_probability = (
    y_train
    .mean()
)

y_prob = np.full(
    len(y_test),
    class_probability
)

metrics = classification_metrics(
    y_test,
    y_pred,
    y_prob
)

majority_results.append({
    "Test Year": w["test_year"],
    "Model": "Majority-Class Baseline",

```

```

        "Accuracy": metrics["Accuracy"],

        "Balanced Accuracy":
            metrics["Balanced Accuracy"],

        "Precision":
            metrics["Precision"],

        "Recall":
            metrics["Recall"],

        "F1":
            metrics["F1"],

        "ROC-AUC":
            metrics["ROC-AUC"],

        "Best Params": {}
    })

majority_df = pd.DataFrame(
    majority_results
)

classification_results_df = pd.concat(
    [
        classification_results_df,
        majority_df
    ],
    ignore_index=True
)

```

```

[49]: classification_summary = (
    classification_results_df
    .groupby("Model")
    [
        [
            "Accuracy",
            "Balanced Accuracy",
            "Precision",
            "Recall",
            "F1",
            "ROC-AUC"
        ]
    ]
    .mean()
    .sort_values(

```

```

        "ROC-AUC",
        ascending=False
    )
)

classification_summary.round(4)

```

```
[49]:
```

	Accuracy	Balanced Accuracy	Precision	Recall	\
Model					
Logistic Regression	0.5046	0.5046	0.5199	0.6832	
XGBoost	0.4946	0.4983	0.5215	0.5440	
Random Forest	0.5010	0.5039	0.5174	0.5887	
Majority-Class Baseline	0.5060	0.5000	0.4222	0.8182	

	F1	ROC-AUC
Model		
Logistic Regression	0.5428	0.5105
XGBoost	0.5168	0.5077
Random Forest	0.5247	0.5027
Majority-Class Baseline	0.5562	0.5000

2 Final Results:

```
[ ]: print("=====")
print("PHASE 1: RAW PRICE REGRESSION")
print("=====")

display(
    raw_summary.round(4)
)

print("=====")
print("PHASE 2: LOG-RETURN REGRESSION")
print("=====")

display(
    log_summary.round(6)
)

print("=====")
print("PHASE 3: DIRECTIONAL CLASSIFICATION")
print("=====")

display(
    classification_summary.round(4)
)

```

)

=====

PHASE 1: RAW PRICE REGRESSION

=====

	MAE	RMSE	R ²
Model			
Naive - Today's Close	0.9798	1.3100	0.9587
Ridge Regression	1.3206	1.6849	0.9294
Linear Regression	1.4051	1.8064	0.9116
Random Forest	6.1396	7.4816	0.1325
XGBoost	6.2295	7.5992	0.1211

=====

PHASE 2: LOG-RETURN REGRESSION

=====

	MAE	RMSE	R ²
Model			
Naive - Zero Return	0.006540	0.008739	-0.004575
XGBoost	0.006580	0.008798	-0.019683
Ridge Regression	0.006596	0.008807	-0.019783
Random Forest	0.006589	0.008809	-0.024235
Linear Regression	0.006822	0.009074	-0.077871

=====

PHASE 3: DIRECTIONAL CLASSIFICATION

=====

	Accuracy	Balanced Accuracy	Precision	Recall	\
Model					
Logistic Regression	0.5046	0.5046	0.5199	0.6832	
XGBoost	0.4946	0.4983	0.5215	0.5440	
Random Forest	0.5010	0.5039	0.5174	0.5887	
Majority-Class Baseline	0.5060	0.5000	0.4222	0.8182	

	F1	ROC-AUC
Model		
Logistic Regression	0.5428	0.5105
XGBoost	0.5168	0.5077
Random Forest	0.5247	0.5027
Majority-Class Baseline	0.5562	0.5000